

AlphaGo Zero 原理介绍

从蒙特卡洛树搜索到自我博弈强化学习

目录



蒙特卡洛树搜索 (MCTS)

博弈树搜索 · UCT公式 · 四阶段流程 · 性质分析



AlphaGo Zero：自我博弈的突破

神经网络架构 · PUCT搜索 · 自我增强 · 训练循环



强化学习基础

MDP · 价值函数 · 蒙特卡洛方法 · 时序差分学习

PART I

蒙特卡洛树搜索

Monte Carlo Tree Search

推荐教程（李理的博客）

- [1. 解析 AlphaGo](#)
- [2. 解析 AlphaGo Zero](#)
- [3. 解析 AlphaZero \(含 MCTS 图解\)](#)

核心论文（附本地 PDF）

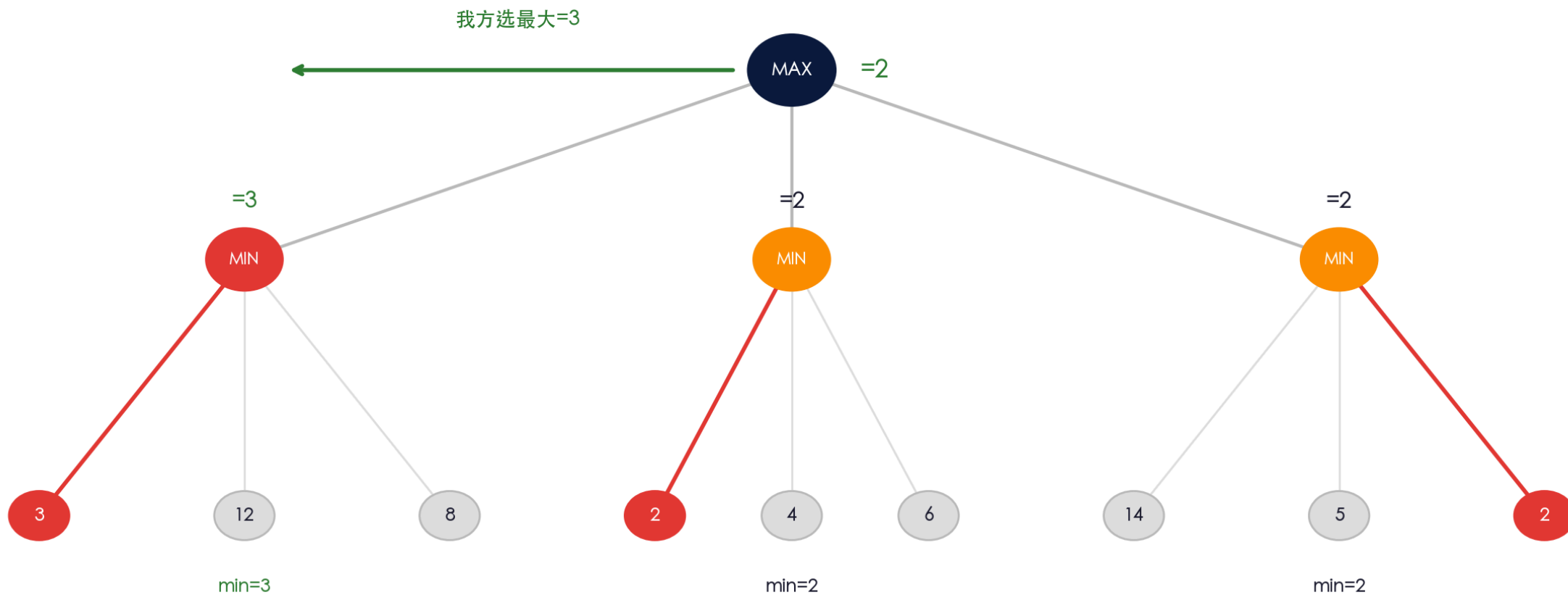
- [AlphaGo \(Nature 2016\)](#)  本地
- [AlphaGo Zero \(Nature 2017\)](#)  本地
- [AlphaZero \(Science 2018\)](#)  本地

Minimax 搜索

最坏情况下最大化收益 (冯·诺依曼)

Minimax 搜索 (冯·诺依曼)

我方最大化 (MAX), 对手最小化 (MIN) — 深度 d 的完整博弈树

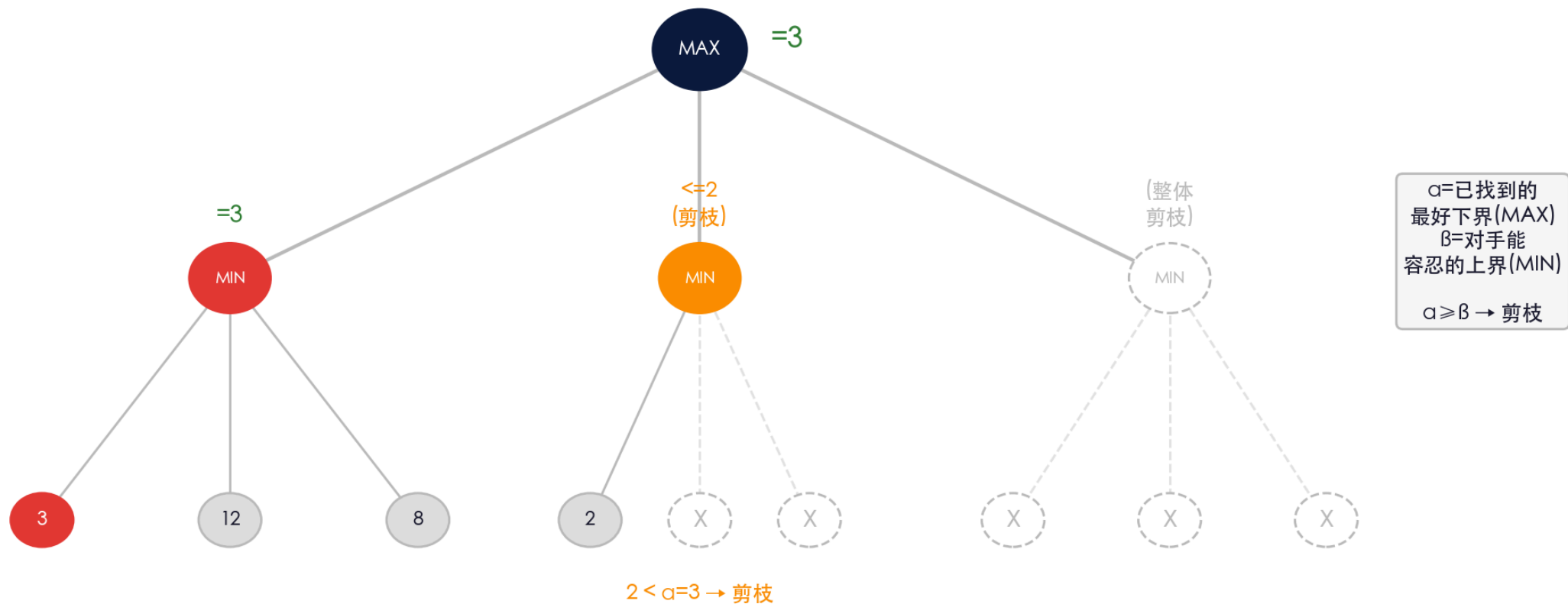


Alpha-Beta 剪枝

剪掉不影响决策的分支

Alpha-Beta 剪枝

剪掉不可能影响决策的分支 — 不改变结果，只减少搜索量



Heuristic 启发式搜索

优先搜子力优势大的走法

Heuristic 启发式搜索：优先搜索好局面



叶节点 = 双方子力差
(简单可计算的估值函数)

→ Minimax 用此估值
搜索深度 6-7 层

Move Ordering (走法排序)

吃车 (+9)	优先搜索
吃马 (+4)	优先搜索
将军	优先搜索
普通走法 (0)	延迟/剪枝
被动防守 (-2)	延迟/剪枝

先搜子力收益大的走法 → α - β 剪枝更高效 (剪得更多)

为什么围棋不能用 Minimax?

分支因子 + 估值函数

为什么围棋不能用 Minimax ?

问题 1: 分支因子太大

国际象棋

分支因子 ~35 深度 6-7 层可搜

围棋 19×19

分支因子 ~250 深度 4 层已极难

$35^6 \approx 18\text{亿}$ vs $250^4 \approx 39\text{亿}$

同样算力下围棋搜索深度远不够

问题 2: 估值函数难写

象棋: 子力价值明确

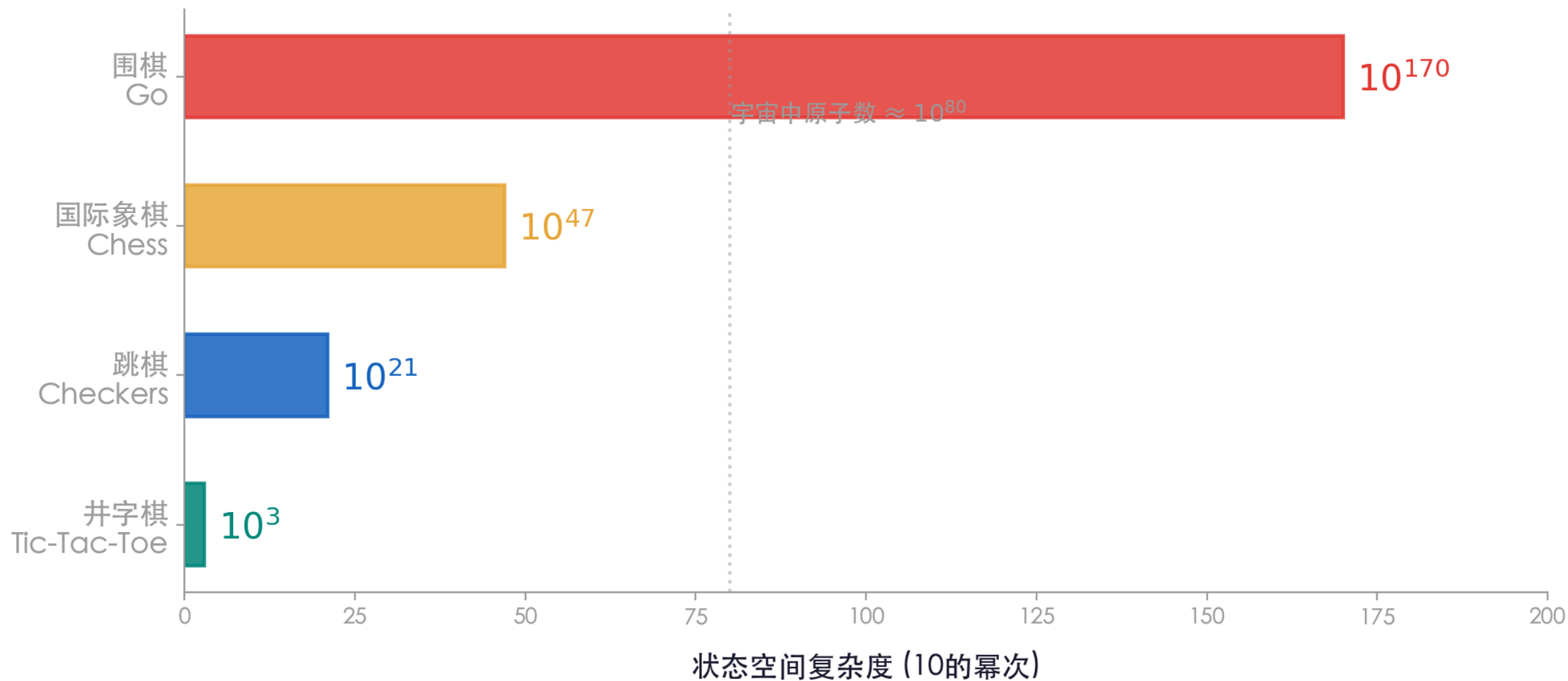
车=9 马/炮=4 兵=1 → 简单求和

围棋: 局面价值极难评估

- 一子的价值随全局形势剧烈变化
- "厚薄""势""实地" 难以量化
- 深层局面并不比浅层更容易评估

→ Minimax 需要好的叶节点估值, 围棋给不了

为什么围棋这么难？



蒙特卡洛方法：从物理到博弈

物理中的蒙特卡洛

- 格点QCD：用MC采样计算路径积分
- Metropolis算法：按 $\exp(-S)$ 采样构型
- 重要性采样：用概率分布引导采样
- 核心思想：用随机采样近似复杂积分

博弈中的蒙特卡洛

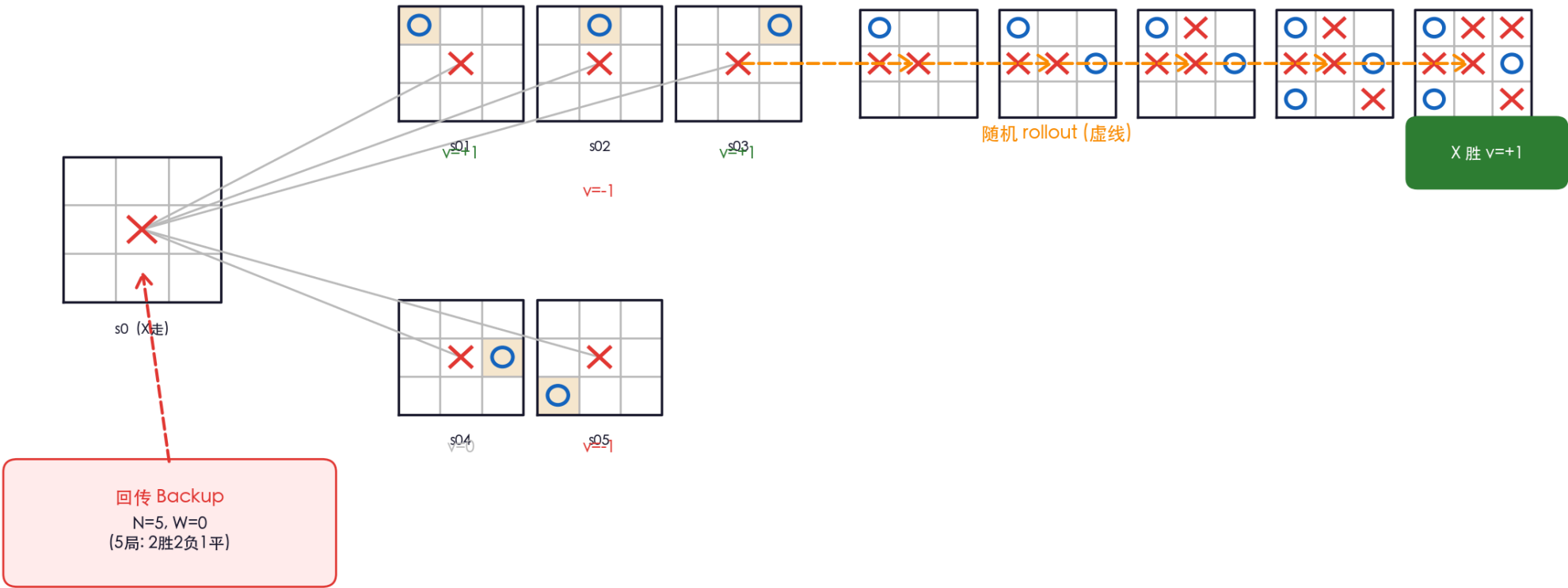
- 随机模拟对局至终局获得结果
- 统计胜率来评估局面好坏
- 用树结构组织搜索方向
- 核心思想：用随机采样近似博弈值

本质相同：用随机采样解决精确计算不可行的问题

传统 MCTS 第 1 次模拟

展开 + Rollout + 回传 (井字棋实例)

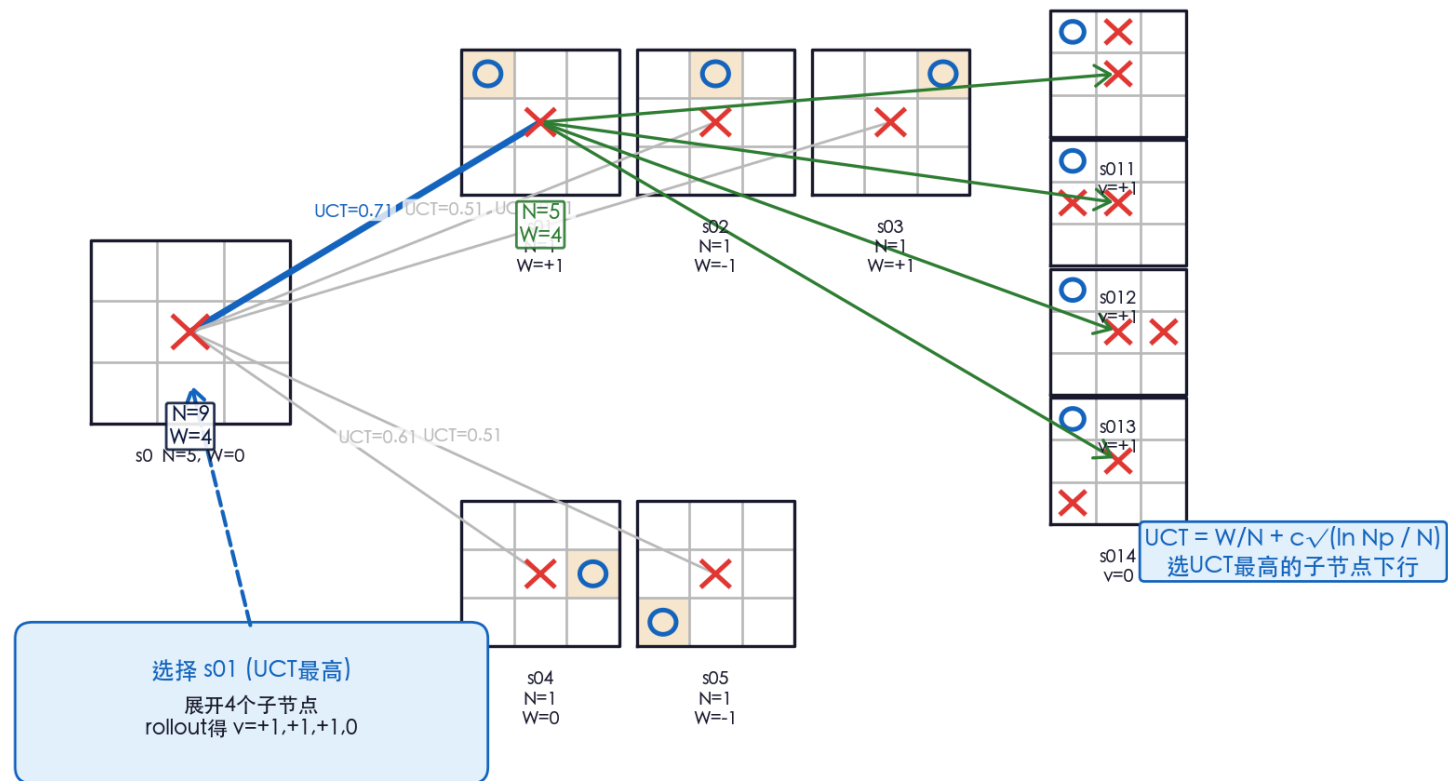
传统 MCTS 第 1 次模拟：展开 + Rollout + 回传



传统 MCTS 第 2 次模拟

UCT 选择 → 展开 → 回传

传统 MCTS 第 2 次模拟：UCT 选择 → 展开 → 回传



搜索树的非对称增长

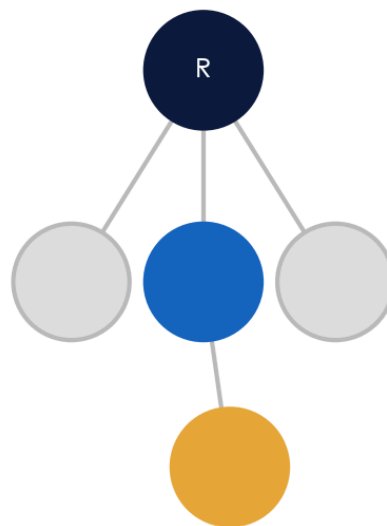
算力集中 = 重要性采样

MCTS 搜索树的非对称增长（算力集中在有价值的分支）

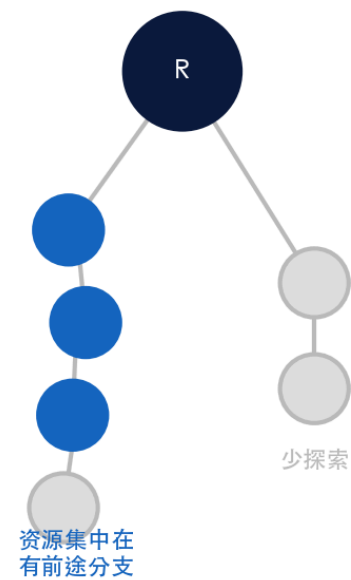
初始（1次模拟）



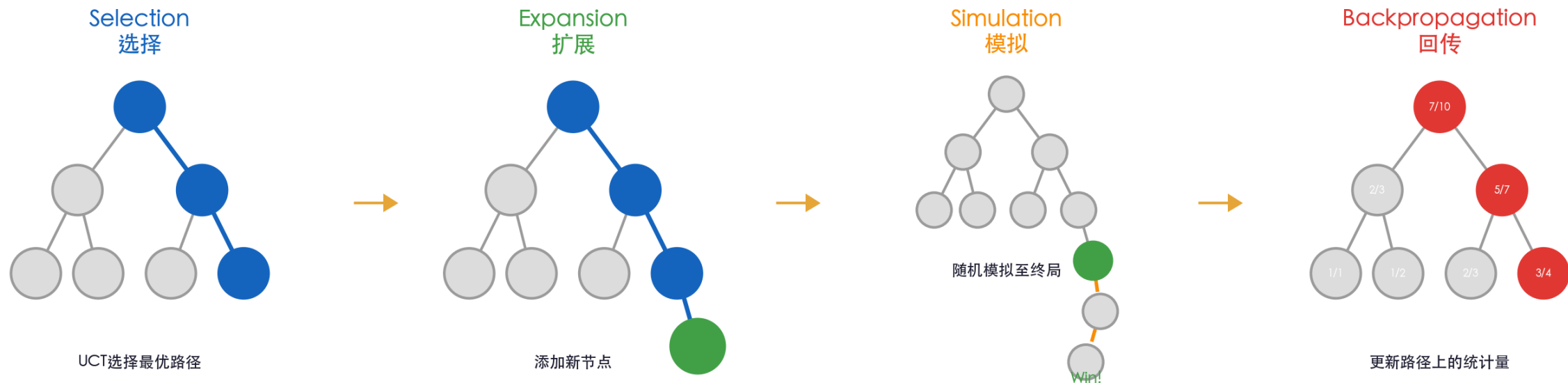
10 次模拟后



100 次模拟后



MCTS 的四个阶段



每次迭代执行这四个步骤，逐步积累统计信息，搜索精度随迭代次数增加而提高

① 选择 Selection

从根节点沿UCT最大的子节点下行

② 扩展 Expansion

到达叶节点后添加一个新子节点

③ 模拟 Simulation

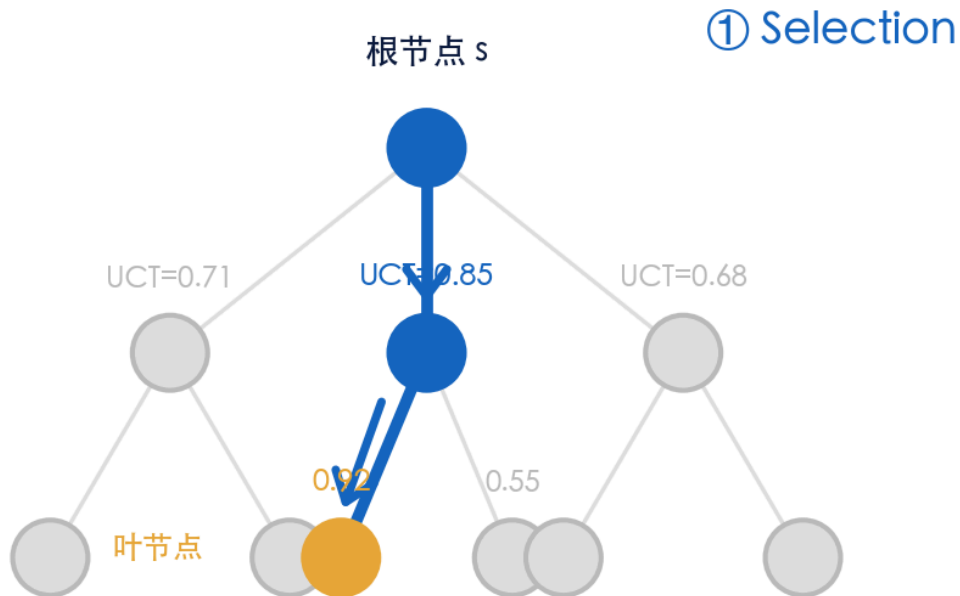
从新节点随机走子直到游戏结束

④ 回传 Backpropagation

将结果沿路径回传更新统计量

① Selection 选择

沿 UCT 最大路径下行至叶节点



从根节点出发
沿 UCT 最大的子节点
递归下行至叶节点

原理

从根节点出发，对每个子节点计算 PUCT 值，选最大者下行，递归至叶节点。

$$u = Q(s, a) + c_{puct} \cdot P(s, a) \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

其中 $Q(s, a)$ 是该边的平均胜率（回传价值 v 的增量平均， $v \in [-1, 1]$ ，故 Q 也落在 $[-1, 1]$ ）。

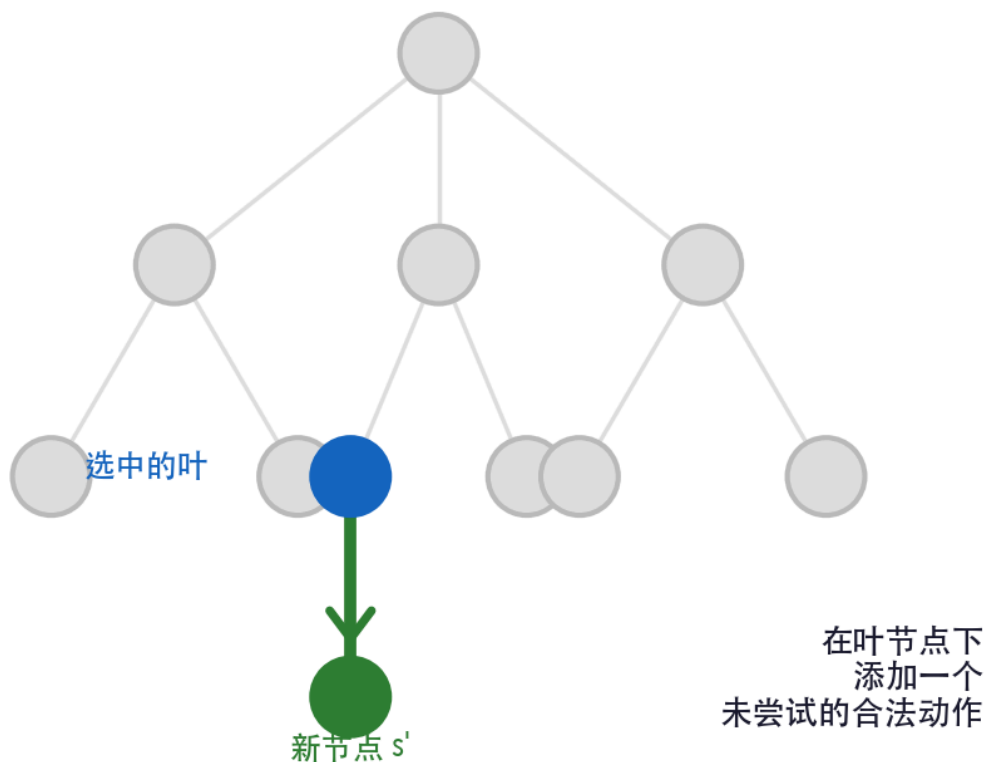
利用-探索平衡： Q 大 $\rightarrow$ 历史好； N 小 $\rightarrow$ 多探索

```
109 for a in range(action_size):
110     if valids[a]:
111         if (s,a) in Qsa:
112             u = Qsa[(s,a)] + cpuct * P[s][a] * sqrt(Ns[s]) / (1 + Nsa[(s,a)])
115             u = cpuct * P[s][a] * sqrt(Ns[s] + EPS) # Q=0
117         if u > cur_best: best_act = a
```

② Expansion 扩展

在叶节点下添加新子节点

② Expansion



原理

到达叶节点后，若非终局，添加一个未尝试的合法动作作为新子节点。

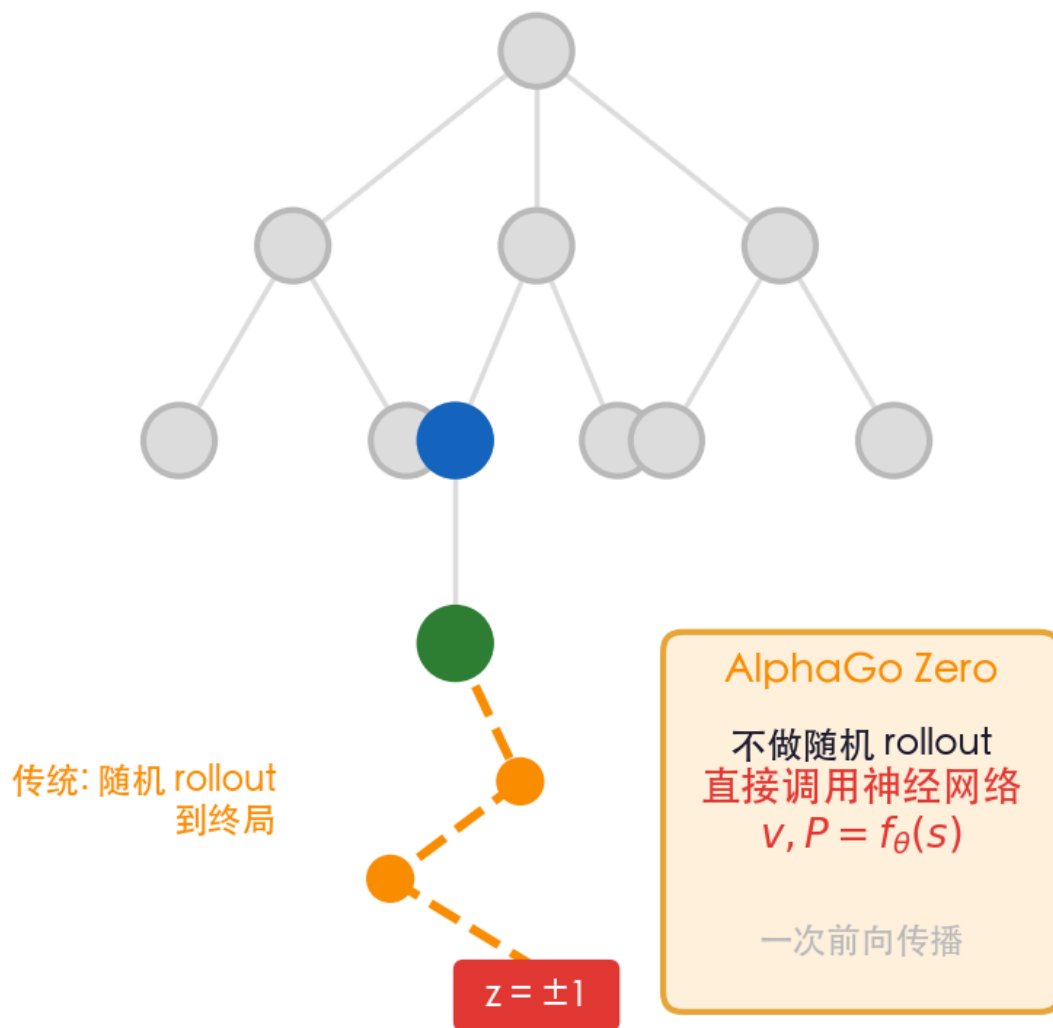
AlphaGo Zero 的特点：**扩展即评估**——首次访问叶节点时立即调用神经网络得到 (P, v) ，扩展与模拟合二为一。

对 P 做合法动作掩码后重归一化（网络可能给非法动作非零概率）。

```
83 if s not in Ps:           # 叶节点
84     Ps[s], v = nnet.predict(board)
85     valids = game.getValidMoves(board, 1)
86     Ps[s] = Ps[s] * valids   # 掩码非法动作
87     Ps[s] /= sum(Ps[s])     # 重归一化
101 Ns[s] = 0
102 return -v                 # 直接回传网络值
```

③ Simulation 模拟 评估新节点价值 (关键差异)

③ Simulation



传统 MCTS vs AlphaGo Zero

传统: 从新节点随机 rollout 到终局, 得 $z = \pm 1$ 。质量差、计算量大 (需上千次模拟)。

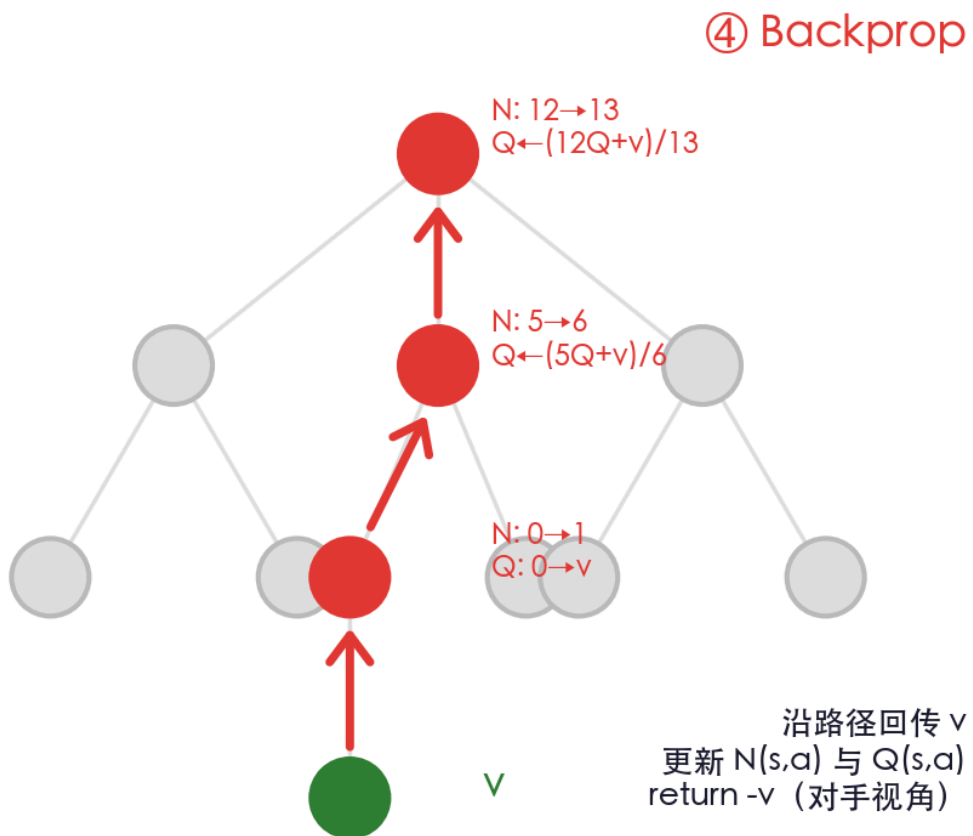
AlphaGo Zero: 不做 rollout。叶节点处直接用网络 $v = f_{\theta}(s)$, 一次前向传播得到价值估计。

→ 论文标题 "without rollouts" 的由来。25 次模拟即可下出好棋。

```
# 传统 MCTS (AlphaGo Zero 不做这一步):  
# while not game_over:  
#     action = random_policy(s)  
#     s = next_state(s, action)  
# return game_result # z = ±1  
  
# AlphaGo Zero: 直接用网络 (MCTS.py:85,102)  
Ps[s], v = nnet.predict(board) # 一次前向传播  
return -v # 替代整个 rollout
```


④ Backpropagation 回传

沿路径更新 N 与 Q



原理

沿选择路径回传，更新每条边的 $N(s,a)$ 与 $Q(s,a)$ ：

$$Q \leftarrow \frac{N \cdot Q + v}{N + 1}, \quad N \leftarrow N + 1$$

关键 1: $W/N = Q$ —— 累计价值除以访问次数即平均值，二者等价。

关键 2: return $-v$ —— 每上一层取负。 W 相对于当前走棋玩家：我的好局面 = 对手的坏局面，这是 minimax 的体现。

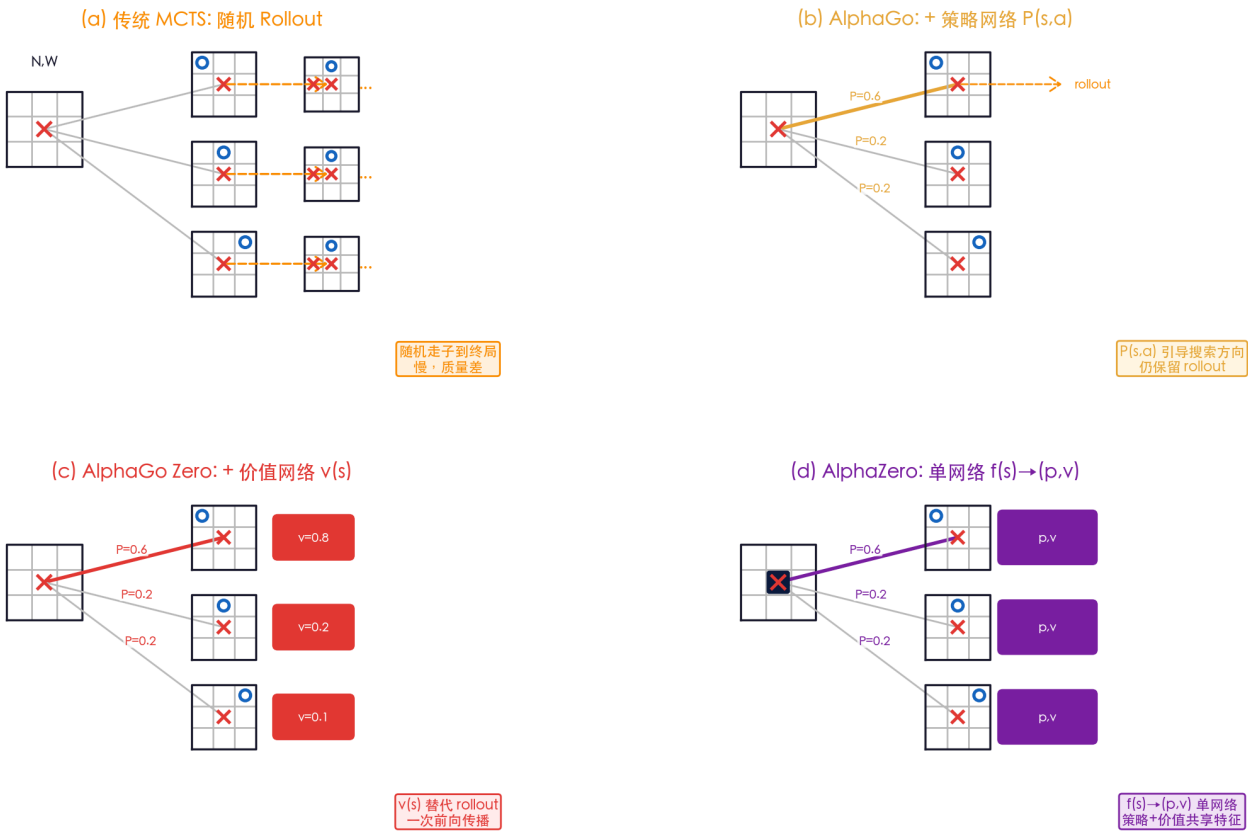
增量平均等价于对 v 求均值， $O(1)$ 空间。本质是 TD 思想。

```
127 if (s,a) in Qsa:
128     Qsa[(s,a)] = (Nsa[(s,a)]*Qsa[(s,a)] + v) / (Nsa[(s,a)]+1)
129     Nsa[(s,a)] += 1
131 else:
132     Qsa[(s,a)] = v; Nsa[(s,a)] = 1
135     Ns[s] += 1
136 return -v # 对手视角取负 (minimax)
```

MCTS 的演进

Rollout → 策略网络 → 价值网络 → 单网络

MCTS 的演进：从随机 Rollout 到 AlphaZero 单网络



随机走到终局
慢，质量差

$P(s,a)$ 引导搜索方向
仍保留 rollout

$v(s)$ 替代 rollout
一次前向传播

$f(s) \rightarrow (p,v)$ 单网络
策略+价值共享特征

(a) 传统 MCTS

随机 rollout 到终局
质量差、上千次模拟
 $U \propto \sqrt{\ln N_p / n_i}$

(b) + 策略网络

$P(s,a)$ 引导搜索方向
好走法优先探索
仍保留 rollout

(c) + 价值网络

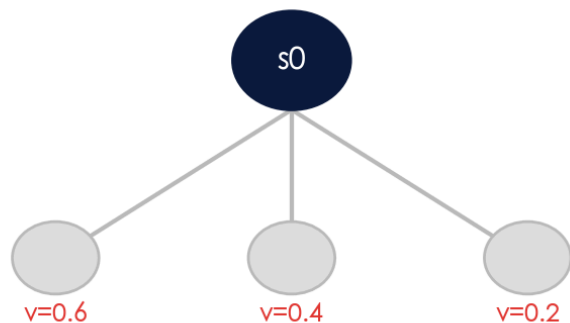
$v(s)$ 替代 rollout
一次前向传播得价值
"without rollouts"

(d) AlphaZero

单网络 $f(s) \rightarrow (p,v)$
策略+价值共享特征
通吃棋类

设计思考 ①：为什么要下行到叶子？只决策一层吗？

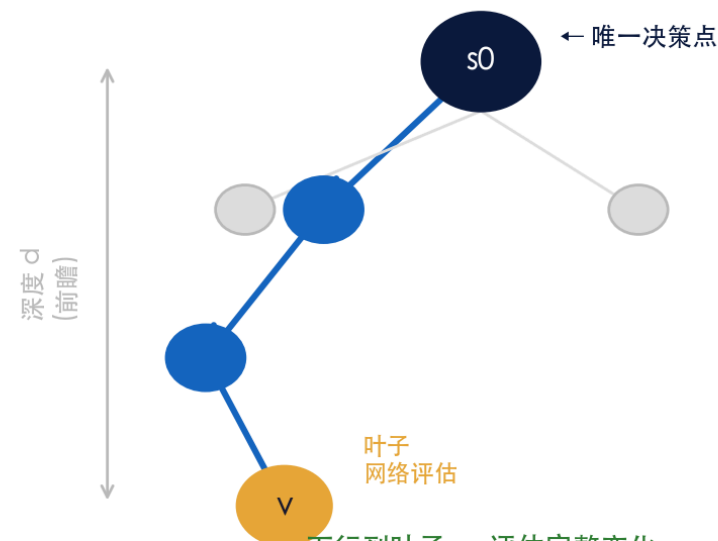
方案 A: 只看一层 (贪心)



只评估直接子节点
无法前瞻 → 短视

X 无 lookahead, 错过陷阱

方案 B: MCTS 下行到叶子



下行到叶子 → 评估完整变化
深层选择=资源调度, 非最终落子

V lookahead, 评估长远后果

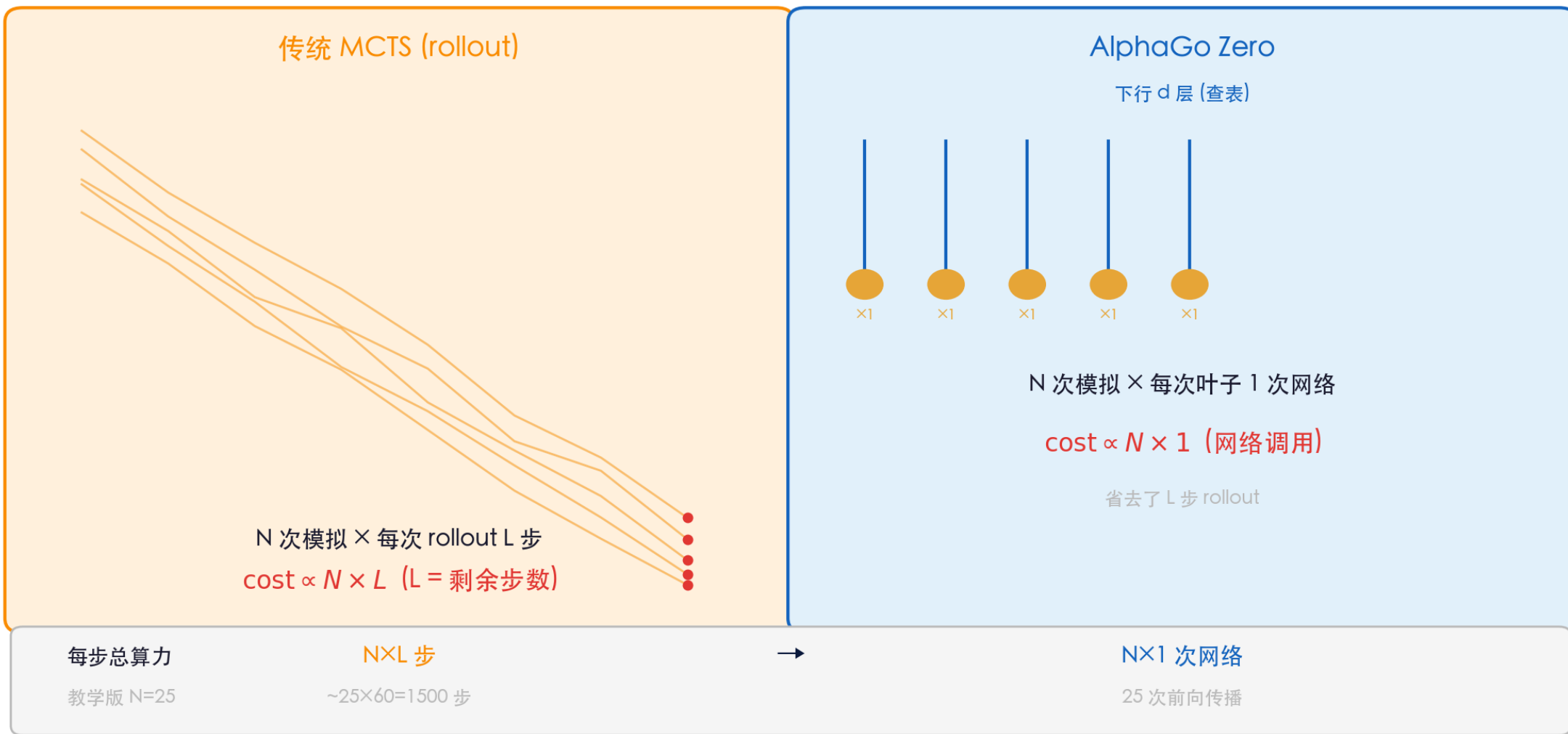
核心洞察

最终决策只在根节点 s_0 一层 (看 $N(s_0, a)$ 分布) ; 下行到叶子是为了把搜索预算分配到有前途的分支, 做 lookahead

设计思考 ②：模拟的复杂度

"只运行一次网络"是什么意思？

每步落子的计算复杂度



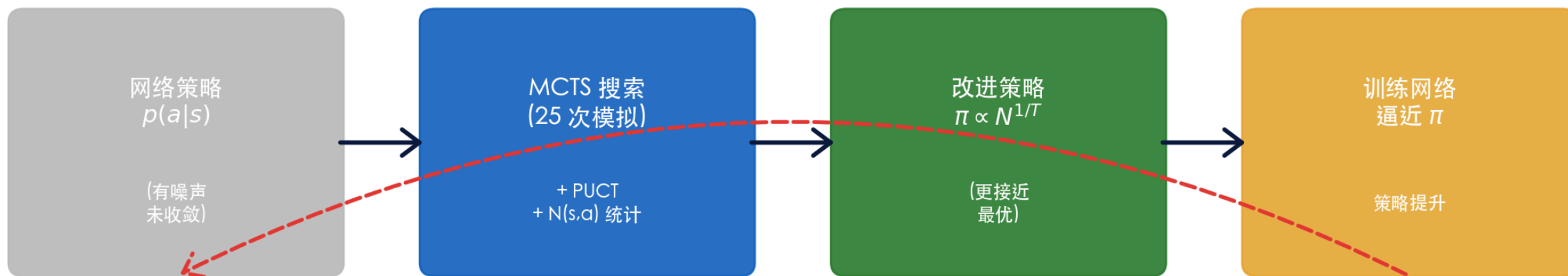
设计思考 ③：MCTS 在系统中的角色

策略改进算子

MCTS = 策略改进算子

关键：MCTS 用搜索把"粗糙的网络 p "改进成"更强的策略 π "，再让网络学 π

这本质是策略迭代 (Policy Iteration) : $\pi_{\text{policy}} \rightarrow \text{MCTS} \rightarrow \pi_{\text{improved}} \rightarrow \text{更新 policy}$



迭代： π 越来越强 $\rightarrow$ 网络越来越准 $\rightarrow$ 搜索越来越好

Selection: UCT 公式

Upper Confidence Bound applied to Trees

$$UCT(i) = \frac{w_i}{n_i} + c\sqrt{\frac{\ln N}{n_i}}$$

利用项 (Exploitation)

w_i/n_i = 第 i 个子节点的平均胜率，偏好好的走法

探索项 (Exploration)

$\sqrt{\ln N/n_i}$: 访问少的节点获得更高的探索奖励

探索常数 c

控制利用与探索的平衡，常用 $c = \sqrt{2}$

核心性质：UCT 在无限采样下收敛到极小极大最优解 (Kocsis & Szepesvári, 2006)

关键辨析：均值无偏差 vs 效率最优 —— 纯随机采样虽有“渐近无偏性”，但因其样本复杂度是指数级的 $O(b^d)$ 导致完全无法实用。UCT 的真正贡献是样本效率最优（最小化累积悔值）：保证次优分支探索仅以对数级 $O(\log N)$ 增长，从而实现非对称高效采样。

类比物理：类似模拟退火中温度控制探索-利用平衡，或 Metropolis 中接受概率的作用

MCTS 的关键性质

渐近/样本效率最优

不同于随机采样高代价的“渐近无偏”，UCT 的真正力量是**样本效率最优**（最小化悔值），保证将资源集中在最优路径。

Anytime 算法

可以随时中断并返回当前最优动作。计算资源越多，决策质量越高。天然适合有时间限制的实时决策。

无需领域知识

不需要人工设计评估函数。仅需知道游戏规则（合法动作和终止条件），即可通过模拟获取价值信息。

自适应搜索

自动将更多计算资源分配给最有前途的分支。搜索树非对称增长，类似重要性采样。

PART II

AlphaGo Zero

自我博弈的突破 · Mastering the Game of Go without Human Knowledge

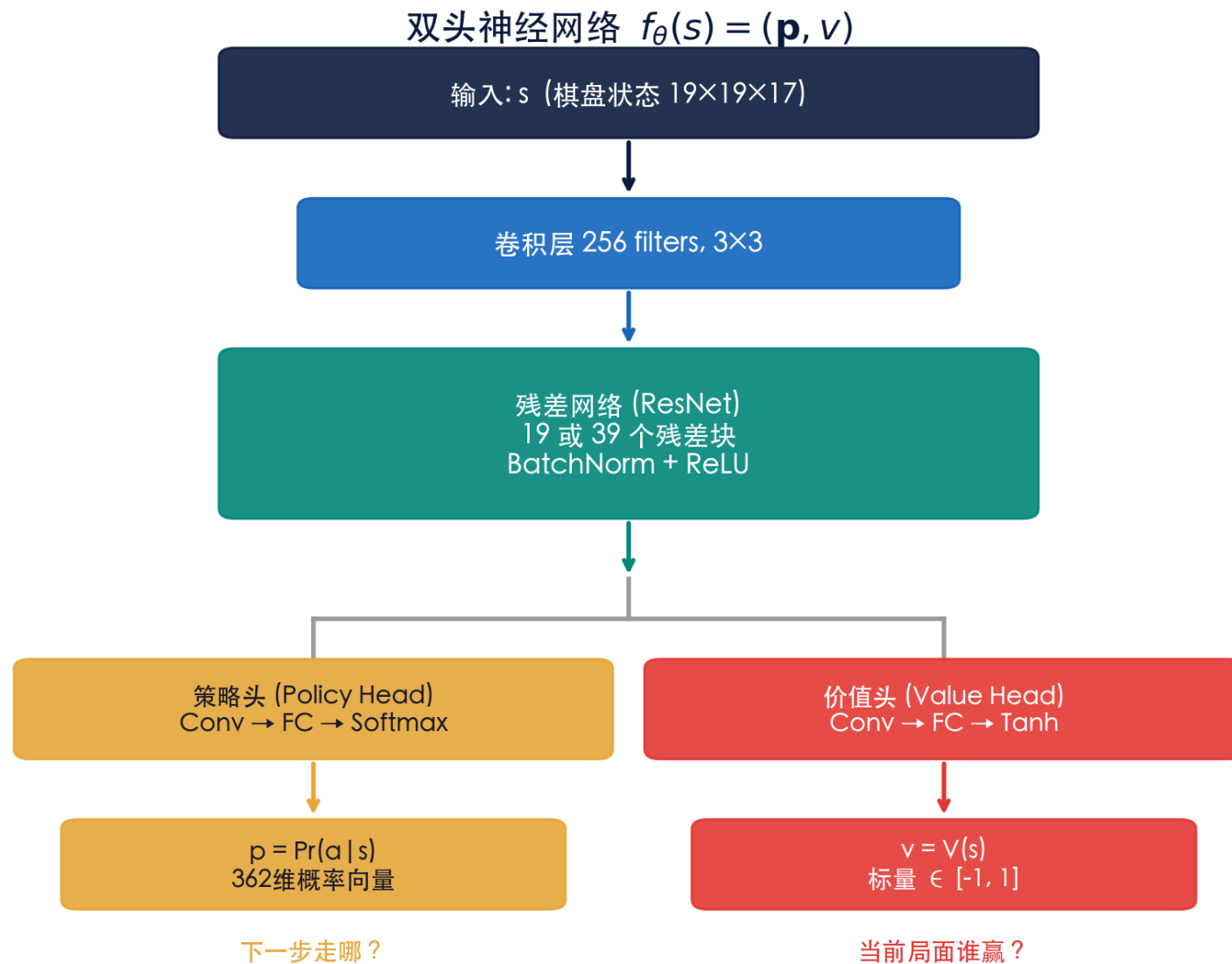
从 AlphaGo 到 AlphaGo Zero



AlphaGo Zero 的三大简化

- ① 不使用任何人类棋谱 (no human data)
- ② 仅用棋盘状态作为输入 (no hand-crafted features)
- ③ 单一神经网络替代策略网络+价值网络

双头神经网络架构 (OthelloNNet.py)



MCTS + 神经网络: PUCT 搜索 (MCTS.py:112-115) · PUCT = Predictor + UCT

MCTS + 神经网络: PUCT 选择公式

$$a^* = \arg \max_a \left[Q(s, a) + c_{\text{puct}} \cdot P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)} \right]$$

$Q(s, a)$

动作价值
(平均回报)

$P(s, a)$

神经网络先验
(策略头输出)

$N(s, a)$

访问次数

← 利用 (Exploitation) →

← 探索 (Exploration) →

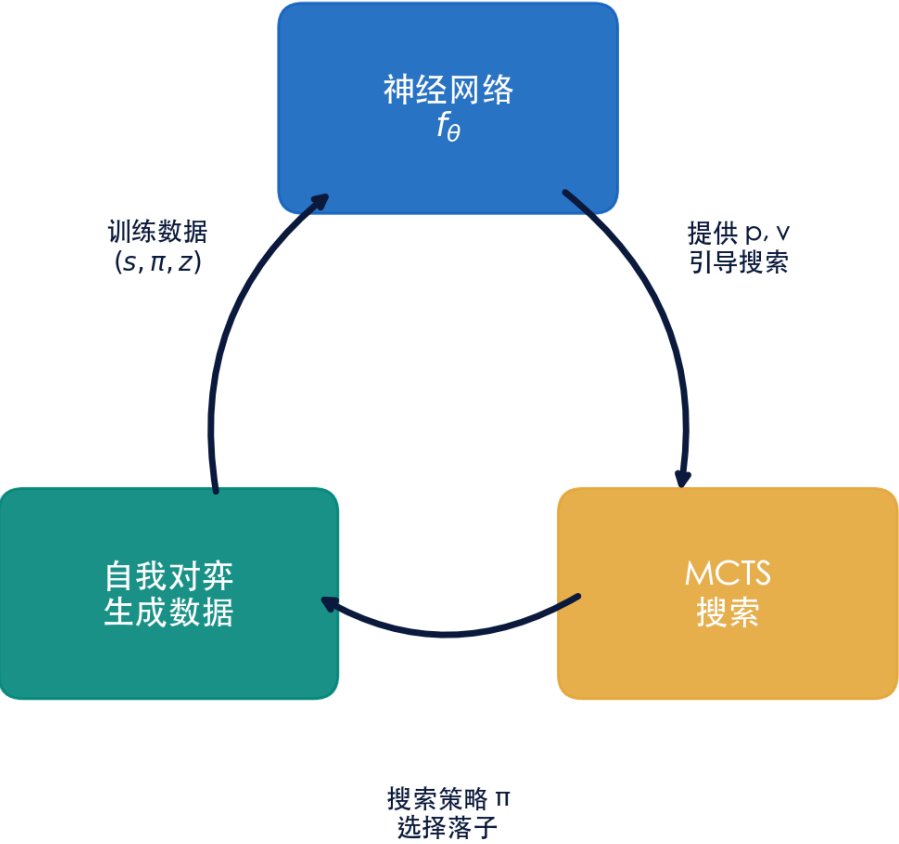
与传统 MCTS 的关键区别

- ① 用 $P(s, a)$ 替代随机模拟 (Simulation) ② 用 $v(s)$ 替代终局结果 ③ 搜索效率大幅提升, 少量模拟即可获得高质量策略

自我对弈与数据生成

MCTS → 落子策略 π → 训练数据 (s, π, z)

AlphaGo Zero 自我学习循环



从搜索到落子：温度采样

MCTS 跑完 N 次模拟后，根节点访问次数 $N(s, a)$ 如何变成落子策略？

$$\pi_i = N(s, a_i)^{1/\tau}$$

对弈 $\tau \rightarrow 0$ ：选 N 最大的（确定性、最强）
自对弈 前期 $\tau = 1$ ：按比例随机采样（探索、防模式坍塌）
教学版 tempThreshold=15：前 15 步探索，之后利用

训练数据 (s, π, z)

- 每步记录：
- s ：棋盘状态
 - π ：MCTS 访问次数分布（温度采样后）
 - z ：终局胜负 (± 1)

一局自对弈 → 一条轨迹 → 多个 (s, π, z)
网络学： $p \rightarrow \pi$ （策略蒸馏）， $v \rightarrow z$ （价值评估）

代码参数对照表 (alpha-zero-general 默认配置)

alpha-zero-general 关键参数对照

MCTS (MCTS.py / main.py)	神经网络 (OthelloNNet.py)	训练 / 自我对弈 (NNet.py / Coach.py)
numMCTSSims 25 每步 MCTS 模拟次数 cpuct 1 PUCT 探索常数 c_{puct} tempThreshold 15 前15步 temp=1，之后 0 PUCT $Q + c \cdot P \frac{\sqrt{N}}{1 + N_{sa}}$ $\sqrt{N}$ 非 $\sqrt{\ln N}$ Q 更新 $\frac{NQ + v}{N + 1}$ 增量平均 动作概率 $\pi \propto N^{1/temp}$ 访问次数温度采样	架构 4 层 CNN 非 ResNet (教学简化) 通道数 512 num_channels 卷积核 3×3 前2层pad=1，后2层无 策略头 →1024→512→A log_softmax 输出 价值头 →1024→512→1 tanh，标量 [-1,1] 正则化 BN + Dropout 0.3 无 L2 权重衰减	numIters 1000 外层迭代轮数 numEps 100 每轮自我对弈局数 lr 0.001 Adam 优化器 epochs / batch 10 / 64 每轮训练 损失 $-\pi \log p + (z-v)^2$ 无 $c \ \theta\ ^2$ arenaCompare 40 新vs旧对弈，≥ 60%接受

所有数值均为 suragnair/alpha-zero-general 默认配置 (Othello 6×6)，对应代码文件已在标题标出

复现算力对比：论文 vs 教学版

AlphaGo Zero 论文		alpha-zero-general (教学)	
(Silver et al., Nature 2017)		(suragnair, Stanford CS238)	
游戏	围棋 19×19	游戏	Othello 6×6
训练时长	3 天 (超 Lee) → 40 天 (最终)	训练时长	~3 天 (80 迭代收敛)
硬件	评估: 单机 4 TPU (Lee 用 48 TPU)	硬件	1 块 NVIDIA Tesla K80
每步 MCTS	1,600 次模拟 (≈0.4s/步)	每轮自我对弈	100 局
网络	20 / 40 残差块, 256 通道	每步 MCTS	25 次模拟
训练数据	700k batch × 2048 (3天版)	网络	4 层 CNN, 512 通道
总自我对弈	3天版 490 万局; 40天版 2900 万局	总自我对弈	~8,000 局
Elo	72h ≈ 5185 (超 Master)	计算量	比论文小 ~6 个数量级

作者原话: "orders of magnitude smaller than the computation used in the AlphaGo paper"

论文参数量级对照 (参数量级 aware)

据 Silver et al. Nature 2017 (AGZ) & Science 2018 (AZ) 正文

AlphaGo Lee	AlphaGo Zero	AlphaZero
2016 · 战李世石	Nature 2017 · 无人类知识	Science 2018 · 通用算法
输入特征	输入特征	输入特征
手工特征 (48 平面)	原始棋盘 (17/19 平面)	原始棋盘 (规则平面)
网络	网络	网络
分离: 策略网 + 价值网 (13 层 CNN 各一)	单一双头 ResNet 20 块(3天) / 40 块(40天)	同一 ResNet 架构, 三棋种通用
MCTS 模拟	MCTS 模拟	MCTS 模拟
随机 rollout 到终局	无 rollout, 用 v(s)	无 rollout, 用 v(s)
训练数据	训练数据	训练数据
人类棋谱 (SL) + 自对弈 (RL)	纯自我对弈 (零人类)	纯自我对弈, 无对称性增强
MCTS 模拟数	MCTS 模拟数	MCTS 模拟数
~数千/步	1,600 / 步 (≈0.4s)	800 / 步 (评估)
硬件	硬件	硬件
分布式, 48 TPU (评估)	评估: 单机 4 TPU	5000 一代 TPU 自对弈 + 16 二代 TPU 训练
训练时长	训练时长	训练时长
数月	3 天 (超 Lee) / 40 天 (超 Master)	9h 象棋 / 12h 将棋 / 13天围棋
Elo (vs 人类)	Elo	搜索吞吐
≈ 3600 (胜李世石)	72h ≈ 5185; 40 天 ≈ 超人	60k pos/s (Stockfish 60M)

关键量级直觉: AGZ 1600 模拟/步 · AZ 800 模拟/步 · 教学版 25 模拟/步 | 搜索量 AZ 仅为 Stockfish 的 1/1000 仍胜

训练目标：损失函数 (NNet.py:96-100)

$$\ell = \underbrace{(z - v)^2}_{\text{价值}} - \underbrace{\pi^\top \log \mathbf{p}}_{\text{策略}} + \underbrace{c \|\theta\|^2}_{\text{L2正则}}$$

注：论文含 $c \|\theta\|^2$ 正则项，教学实现省略（靠 BN + Dropout 防过拟合）

价值损失 $(z - v)^2$

z: 自我对弈终局结果 (± 1)

v: 网络预测的价值 (tanh)

`loss_v = sum((target-output)**2)/N`

让网络学会准确评估局面

策略损失 $-\pi^\top \log p$

π : MCTS 访问次数分布

p: 网络输出 `log_softmax`

`loss_pi = -sum( $\pi \cdot \log p$ )/N`

本质是交叉熵 / 知识蒸馏

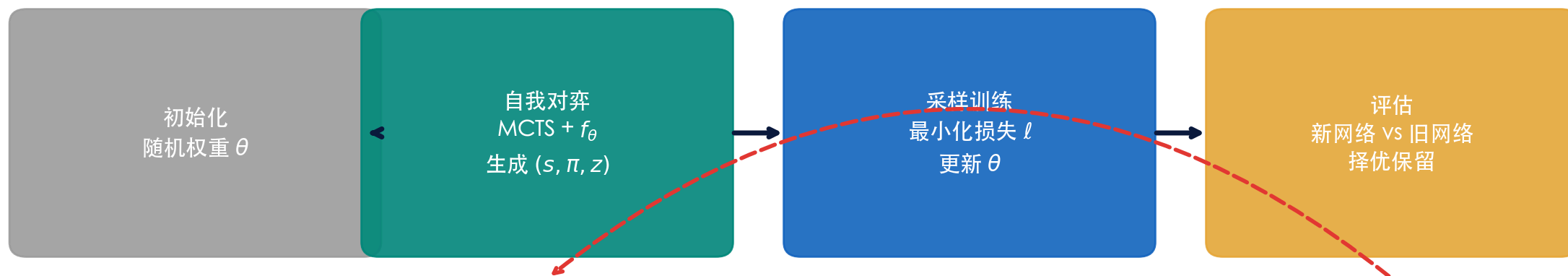
策略提升的闭环 (Coach.py)

MCTS搜索 $\rightarrow$ 得到改进策略 $\pi \rightarrow$ 训练网络逼近 $\pi \rightarrow$ 网络指导MCTS $\rightarrow$ 更强的搜索 $\rightarrow \dots$

类比物理中的自洽场方法 (SCF): 每轮迭代都以前一轮结果为基础, 逐步逼近最优解

完整训练流程 (Coach.py: learn)

AlphaGo Zero 训练流程



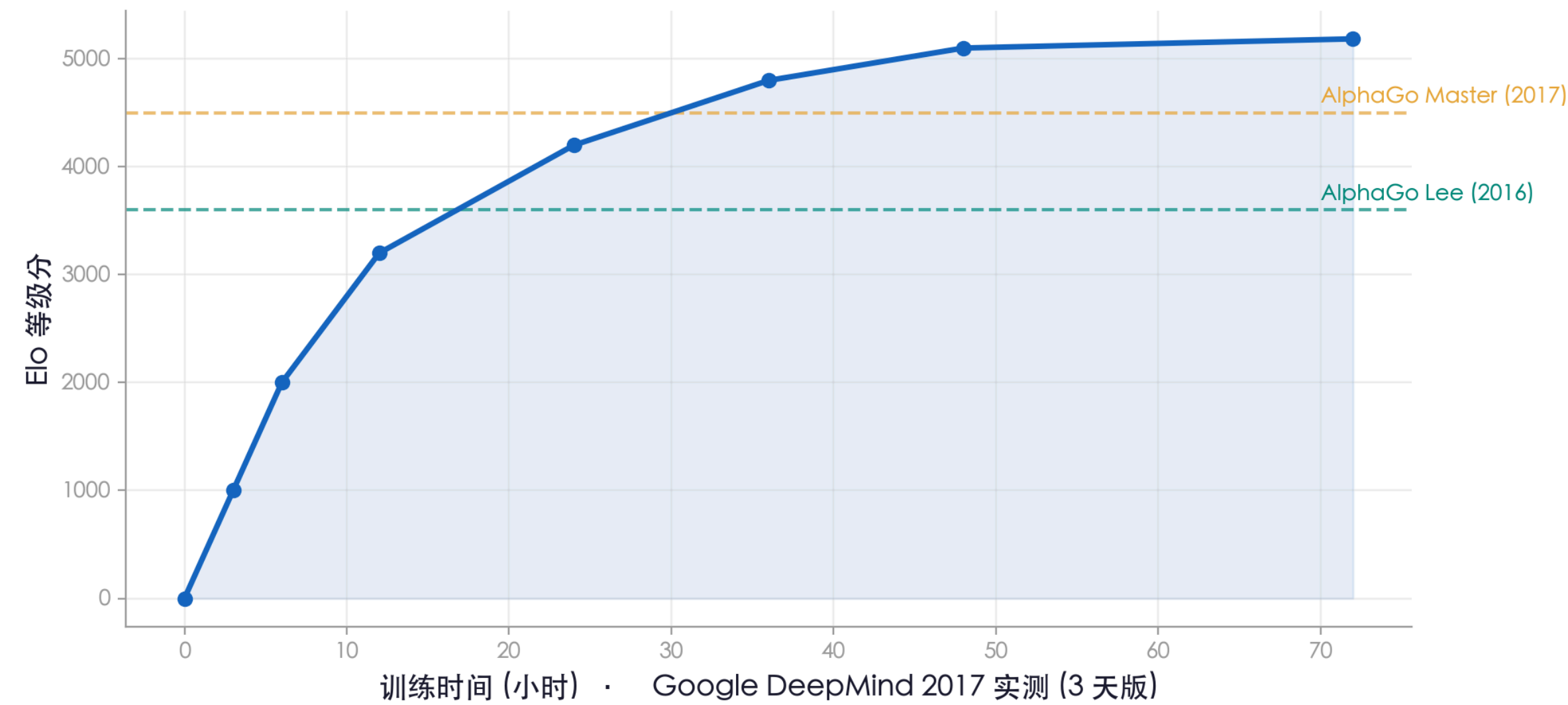
迭代循环：越来越强

教学版：1 块 K80 GPU, ~3 天, 80 迭代收敛 | AGZ：评估 4 TPU, 1600 模拟/步, 3 天 490 万局

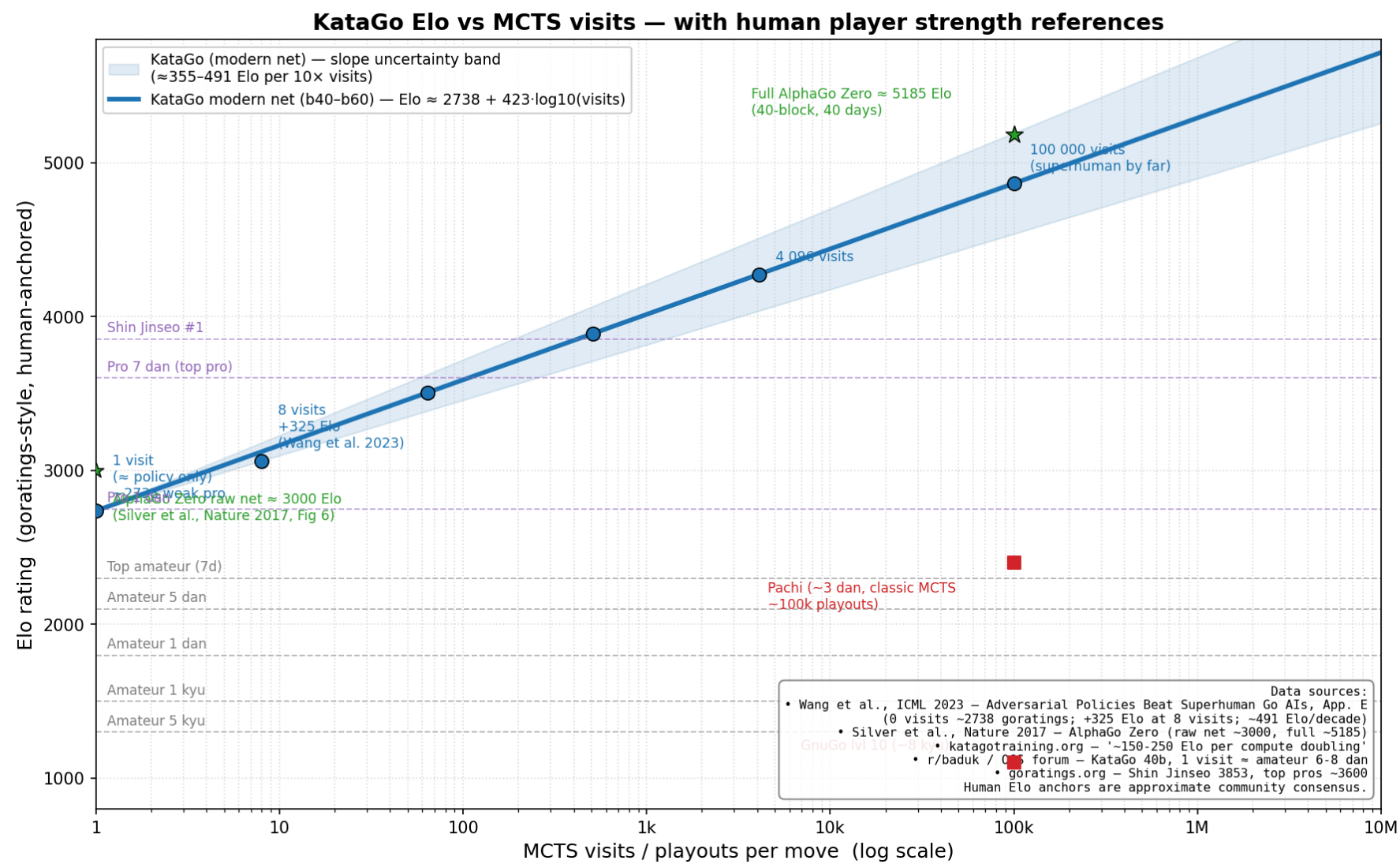
并行计算架构：自我对弈各 worker 独立，天然适合大规模并行加速（HPC 场景）

训练效果

AlphaGo Zero 训练过程中的棋力增长



KataGo 强度 vs MCTS 搜索量 (对数线性标度律)



$\text{Elo} \approx \text{Elo}_0 + k \cdot \log_{10}(\text{visits})$ | 每搜索量 $\times 10 \approx +350\text{--}490$ Elo | 1 visit $\approx$ 业余 6–8 段

KataGo 核心算法优化：超越 AlphaZero 的秘诀

相比于 AlphaGo Zero, KataGo (Wu 2019) 引入了多项关键改良, 将训练效率提升了 50~100 倍

1. 多任务辅助训练 (Auxiliary Targets)

- 目数预测 (Score Lead): 预测具体的终局输赢目数差。
- 子力归属预测 (Ownership): 预测棋盘上每个交叉点最终归属。
- 对手下一步预测: 强迫网络学习站在对手角度思考。
- 效果: 建立围棋空间概念, 极快缩短完全零基础的摸索期。

2. 变化算力自弈 (Playout Cap Randomization)

- 非对称模拟分配: 约 90% 的常规步仅用极低模拟量 (如 100 次以下) 自弈; 仅在 10% 的关键点使用完整模拟量 (如 800 次) 生成高质量策略标签。
- 效果: 极大地节省了自弈所需的计算代价, **自弈数据生成效率提升数倍**。

3. 全局感知 SE 模块 (Global Squeeze-and-Excitation)

- 全局通道特征提取: 在网络中引入 SE 模块与全局池化, 打破了经典残差 CNN 只能在多层堆叠后才能传导全局大局观信息的物理局限。
- 效果: 显著提升网络对“长距离”棋子关联、死活和全局厚势的直觉。

4. “胜率-目数”混合效用函数

- 克服 AlphaZero 的弊端: AlphaZero 只看胜率, 优胜时易下“退让手”造成目数亏损, 劣势时易乱下; KataGo 追求目数优势最大化。
- 效果: 行棋策略更拟人、更稳健, 在让子棋 (Handicap Games) 和逆风局中表现极强。

💡 规则兼容与工程大众化革新

直接将贴目 (Komi) 和规则信息 (中/日/韩) 作为特征输入, 支持单模型全规则兼容; 引入 OpenCL/Vulkan/TensorRT, 使普通家用显卡 (而非昂贵的 TPU 集群) 也能高效运行和训练超人类强度的围棋 AI。

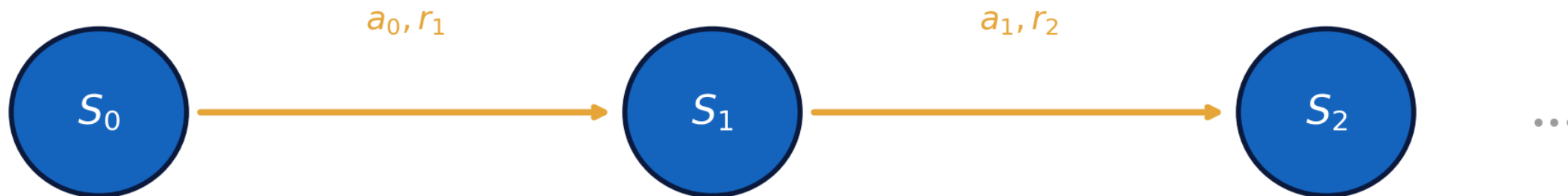
PART III

强化学习基础

Reinforcement Learning Foundations

马尔可夫决策过程 (MDP)

马尔可夫决策过程 (MDP)



$$\mathcal{M} = \langle S, \mathcal{A}, P, R, \gamma \rangle$$

S : 状态空间

$\mathcal{A}$: 动作空间

P : 转移概率

R : 奖励

γ : 折扣因子

围棋天然是一个 MDP: 状态 = 棋盘局面, 动作 = 落子位置, 奖励 = 终局胜负 (± 1)

RL 的目标: 学习最优策略 π^* 使得累积奖励最大

价值函数与策略

策略 $\pi(a|s)$

在状态 s 下选择动作 a 的概率分布

策略是 agent 的"行为准则"

目标：找到最优策略 π^*

状态价值函数 $V(s)$

从状态 s 出发，遵循策略 π 所能获得的期望累积回报

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

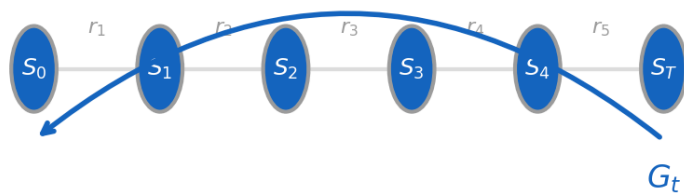
动作价值函数 $Q(s, a)$

在状态 s 执行动作 a 后，遵循策略 π 的期望累积回报： $Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$

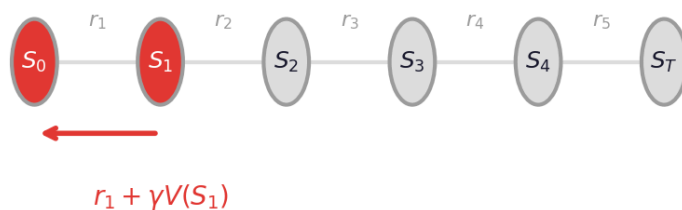
在 AlphaGo 中: $Q(s, a)$ 就是 MCTS 节点统计的平均回报，用来指导选择最优走法

蒙特卡洛方法 vs 时序差分学习

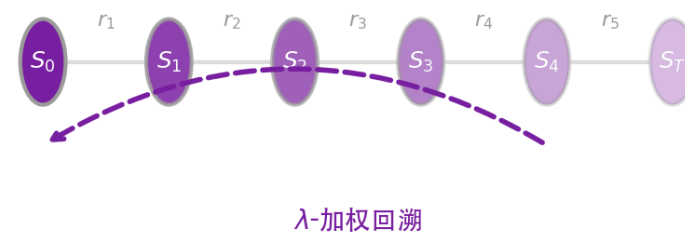
蒙特卡洛 (MC)



TD(0)



TD(λ)



MC 更新：等到终局

$$V(S_t) \leftarrow V(S_t) + \alpha[G_t - V(S_t)]$$

无偏但高方差，必须完成完整轨迹

TD(0) 更新：每步即学

$$V(S_t) \leftarrow V(S_t) + \alpha[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]$$

有偏但低方差，用估计值 bootstrap

TD(λ): MC 与 TD 的统一框架

TD(0) $\lambda=0$

TD(λ) $\lambda=0.5$

MC $\lambda=1$

n-step 回报

1-step: $G^{(1)} = R_{t+1} + \gamma V(S_{t+1}) \leftarrow \text{TD}(0)$

2-step: $G^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2})$

n-step: $G^{(n)} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^n V(S_{t+n})$

∞ -step: $G^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \cdots \leftarrow \text{MC}$

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G^{(n)}, \quad \lambda \in [0, 1] \text{ 控制看多远}$$

总结

MCTS

通过蒙特卡洛模拟在博弈树中进行高效的选择性搜索

UCT 平衡利用与探索

无需评估函数

RL 基础

TD 学习：每步更新

结合估计与真实回报

从 MC 到 TD 的统一框架 $TD(\lambda)$

AlphaGo Zero

神经网络替代模拟

MCTS 提供策略改进

自我对弈生成数据

从零开始超越人类

核心洞察

搜索 (MCTS) + 评估 (Neural Network) + 学习 (Self-Play RL) = 从零开始的超人智能

类比物理：就像变分蒙特卡洛 (VMC) —— 用参数化的试探波函数 (神经网络) 引导蒙特卡洛采样 (MCTS)，然后通过优化变分参数 (训练) 来逼近基态 (最优策略)

谢谢！